

PROGRAMACIÓN II

Trabajo Final Integrador (TFI)

Aplicación Java con relación 1→1 unidireccional + DAO + MySQL

Profesor: Alberto Cortez

Tutor: Marcos Vega

Estudiantes Grupo 98:

Belen Grosso - Comisión 15 - belugrosso98@gmail.com

Natalia Córdoba - Comisión 7 - nataliacorrdoba@gmail.com

Rocío Moyano - Comisión 18 - rocio.moyano@tupad.utn.edu.ar

Yohanna Díaz Monroy – Comisión 13 - yohanna.diaz@tupad.utn.edu.ar

Índice

Introducción.....	3
Etapla 1: Diseño del Sistema (previo al código).....	4
Etapla 2: Entidades (modelo genérico).....	5
Etapla 3: Base de Datos (MySQL).....	6
Etapla 4: Capa DAO.....	7
Etapla 5: Capa Service.....	8
Etapla 6: AppMenu (interfaz por consola).....	9

Link al video:

<https://drive.google.com/file/d/1TRLh1JREfzsuzqFaLv-63C71E4lyCRTc/view?usp=sharing>

Link del repositorio Git Hub:

<https://github.com/Natalia-Cordoba/Trabajo-Practico-Integrador-Programacion2-UTN>

Introducción

El siguiente trabajo explica cómo desarrollamos un sistema de gestión de mascotas y microchips, desde su diseño inicial hasta su implementación.

Lo que buscamos fue realizar una aplicación robusta que manejara la información de forma confiable y segura. En este sistema, trabajamos con dos entidades (mascotas y microchips) que mantienen una relación de asociación entre ellas (1→1).

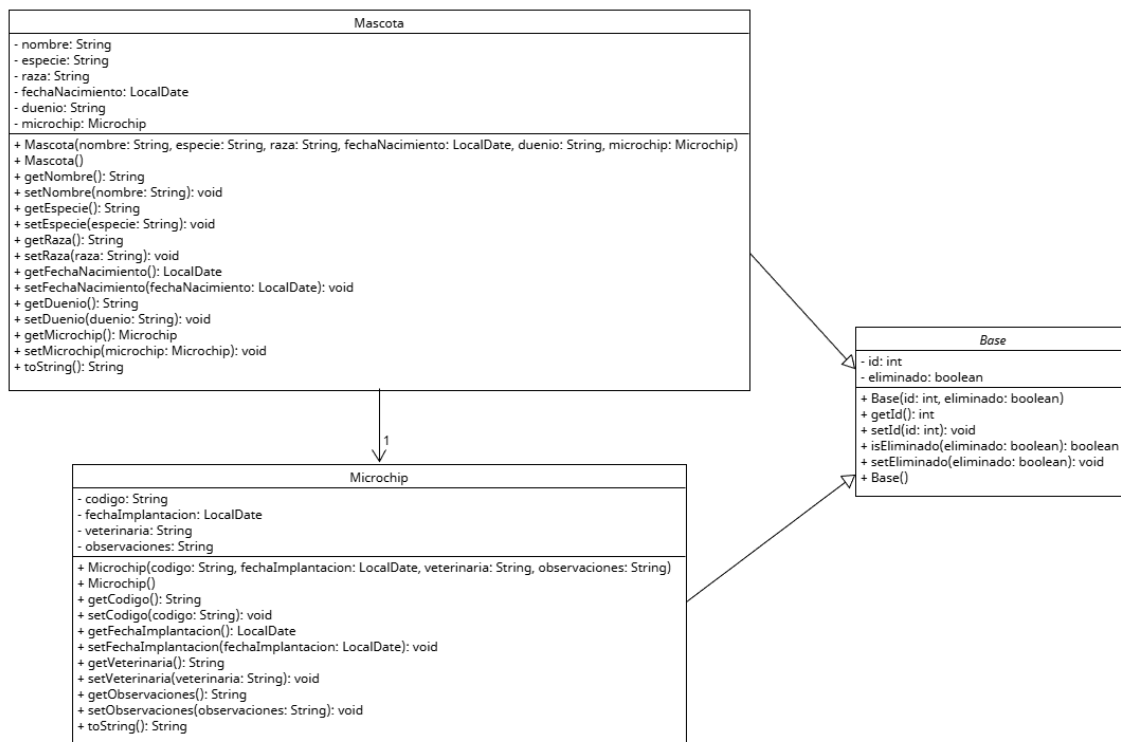
Para realizar este proyecto, utilizamos una serie de herramientas: para desarrollar la aplicación trabajamos con el lenguaje de programación Java; para almacenar, organizar y recuperar los datos usamos el sistema de gestión de bases de datos relacionales MySQL y, por último; para establecer la conexión entre la aplicación y la base de datos, hicimos uso de JDBC.

Para desarrollar el sistema, seguimos el patrón de arquitecturas por capas, en este caso, separamos la lógica en las capas DAO, Service y AppMenu que nos permitieron trabajar de forma ordenada y modular.

Etapa 1: Diseño del Sistema (previo al código)

La siguiente imagen contiene el diagrama UML que realizamos para diseñar el paquete models de nuestro proyecto.

Este diagrama muestra cómo están organizadas las clases, qué atributos tienen y cómo se relacionan entre sí. La clase Base (en cursiva) es una clase abstracta que no puede ser instanciada pero actúa como padre común de las entidades Mascota y Microchip, las cuales toman su funcionalidad y agregan sus datos propios. También vemos la relación entre Mascota y Microchip: una mascota puede tener o no un microchip.



Etapa 2: Entidades (modelo genérico)

En el paquete `models` se definen tres entidades que representan el modelo de dominio: `Base`, `Mascota` y `Microchip`. `Base` provee la estructura común de identidad y estado con los atributos `id` (`int`) y `eliminado` (`boolean`), constructores por defecto y con parámetros, y los getters/setters necesarios.

La clase `Mascota` extiende `Base` y modela una mascota con los atributos `nombre`, `especie`, `raza`, `fechaNacimiento` (`java.time.LocalDate`), `duenio` y una asociación de asociación opcional a `Microchip` mediante el atributo `microchip`. Dispone de un constructor por defecto y de un constructor completo (con todos los atributos del modelo) que invoca `super(id, false)` inicializando el registro como “activo”. Los setters de tipo `String` validan que el valor no sea nulo ni vacío antes de asignarlo; el setter de fecha acepta una cadena en formato ISO (`AAAA-MM-DD`), intenta parsearla a `LocalDate` y lanza `IllegalArgumentException` con el mensaje “Formato de fecha inválido. Use AAAA-MM-DD.” en caso de error. El método `toString()` incluye los campos y la información del `microchip`.

La clase `Microchip` también extiende `Base` y contiene los atributos `codigo` (identificador), `fechaImplantacion` (`LocalDate`), `veterinaria` y `observaciones`. Dispone de un constructor por defecto y de un constructor con parámetros que llama a `super(id, false)`; Los setters de `String` validan no ser nulos ni vacíos y el setter de fecha es igual al de `Mascota`. El `toString()` muestra sus campos.

A nivel de integración, las tres clases comparten paquete y patrón de herencia para reutilizar `id` y `eliminado`; las fechas se manejan con `java.time.LocalDate` y se aceptan mediante cadenas en formato `AAAA-MM-DD` en los setters que lo permiten, por lo que al persistir en SQL será necesario convertir entre `LocalDate` y el tipo de fecha de la base de datos. El borrado lógico debe gestionarse mediante el flag `eliminado` proporcionada por `Base`.

Etapa 3: Base de Datos (MySQL)

El repositorio incluye dos archivos .sql: creaciondb.sql y datos_de_prueba.sql.

El primer archivo contiene todos los SQL statements para crear la base de datos desde cero y es idempotente: cada vez que se ejecuta se obtiene el mismo resultado; esto es posible ya que se incluyen los siguientes comandos:

SET FOREIGN_KEY_CHECKS = 0; Para desactivar temporalmente las comprobaciones de integridad referencial

DROP TABLE IF EXISTS `mascota`; DROP TABLE IF EXISTS `microchip`;
Eliminar tablas si existen

SET FOREIGN_KEY_CHECKS = 1; Reactivar comprobaciones de integridad referencial

CREATE TABLE IF NOT EXISTS Microchip ... IF NOT EXISTS solo crea la tabla si no existe en la DB.

El script datos_de_prueba.sql contiene 5 mascotas con 5 microchips asociados. Todos los registros están activos, ninguno aparece como eliminado.

Todas las mascotas tienen microchip y todos los microchips están referenciados en estas mascotas.

Una vez ejecutados los statements del script, se puede comenzar las pruebas del programa listando, modificando y eliminando los registros. A su vez también el programa permite la creación de nuevas instancias y la asociación entre ellas.

Etapa 4: Capa DAO

En esta parte del trabajo, creamos una interfaz llamada GenericDAO y dos clases, MascotaDAO y MicrochipDAO, que la implementan.

La interfaz GenericDAO nos sirvió para definir los métodos básicos de las operaciones CRUD (crear, leer, actualizar y eliminar) que se realizan sobre cada entidad. Además sobrecargamos estos métodos para que funcionen tanto con una conexión interna, como con una conexión externa (dentro de una misma transacción) para no repetir código.

Después, las clases MascotaDAO y MicrochipDAO, usan los métodos CRUD que definimos en la interfaz para acceder a los datos de cada entidad. Estas clases usan PreparedStatement para evitar problemas de seguridad con SQL y delegan la realización de la tarea a los métodos que reciben la conexión. También tienen métodos auxiliares, como:

- setMascotaParameters y setMicrochipParameters: para cargar los parámetros comunes de cada entidad en el PreparedStatement.
- setGeneratedId: para obtener el ID generado automáticamente por la base de datos.
- mapResultSetToMascota y mapResultSetToMicrochip: para transformar un ResultSet en objetos.
- setMicrochipId: para verificar si una mascota tiene un microchip válido antes de guardar.

En MascotaDAO, realizamos los métodos CRUD y también relacionamos a las mascotas con sus microchips llamando a MicrochipDAO con la misma conexión. En MicrochipDAO, realizamos los métodos CRUD y agregamos un método más para buscar un microchip por código. Esto lo usamos en la capa Service para asegurarnos de que los códigos sean únicos.

Organizamos la capa DAO para que sea fácil de mantener, de ampliar y de controlar las transacciones, separando el acceso a los datos del resto del programa.

Etapa 5: Capa Service

Capa de Negocio (Service) y Transaccionalidad

La Capa de Servicio fue implementada para aplicar la lógica de negocio y asegurar la integridad de los datos. Actuando como coordinador, esta capa garantiza la consistencia de las operaciones complejas mediante el manejo explícito de transacciones.

Puntos Clave

- Estructura: Se definieron las interfaces `GenericService` y las implementaciones específicas (`MascotaService` y `MicrochipService`).
- Atomicidad Garantizada (`MascotaService`):
 - Para operaciones compuestas (crear `Mascota` y `Microchip`), se utiliza una conexión compartida en el constructor.
 - Se deshabilita el *autocommit* (`setAutoCommit(false)`).
 - Las operaciones sólo se confirman (`commit()`) si ambas tienen éxito.
 - Cualquier error provoca la reversión total (`rollback()`), evitando datos inconsistentes.
 - El *autocommit* se restablece en el bloque `finally`.
- Reglas de Negocio:
 - Se aplican validaciones de dominio (campos obligatorios y límites de longitud) en ambos servicios.
 - `MascotaService` hace cumplir la regla 1→1 (un `Microchip` por `Mascota`).
 - `MicrochipService` asegura la unicidad del código del microchip antes de la persistencia.

En esencia, la capa Service es el controlador de integridad que envuelve la lógica empresarial y las interacciones con el DAO en transacciones atómicas.

Etapa 6: AppMenu (interfaz por consola)

El módulo de menú desarrollado funciona como el punto de entrada y el orquestador principal del sistema de gestión de Mascotas y Microchips. Su propósito es administrar la interacción inicial con el usuario, organizar la navegación entre las distintas funcionalidades y delegar acciones hacia los componentes especializados del sistema.

La solución se estructura en tres clases: AppMenu, responsable del ciclo de vida de la aplicación y del manejo centralizado del Scanner; MenuDisplay, encargada exclusivamente de mostrar las opciones y menús en pantalla; y MenuHandler, que interpreta las elecciones del usuario y dirige el flujo hacia los submenús correspondientes. Esta división garantiza una arquitectura clara, ordenada y fácil de mantener.

El módulo permite al usuario gestionar operaciones esenciales del dominio: registrar, consultar, listar y eliminar tanto Mascotas como Microchips, organizadas en submenús independientes que mejoran la usabilidad y la coherencia visual.

En síntesis, el módulo de menú ofrece una interfaz textual simple y eficiente, asegura un flujo de navegación robusto y mantiene una clara separación entre presentación, control y lógica de negocio, contribuyendo de forma significativa a la calidad y escalabilidad del sistema completo.